

LangChain Tutorial

Theory, Patterns & Real-World Applications

langchain-core 1.2.16 | langchain 1.2.10 | Pydantic v2

Version Notice: langchain-core 1.x vs 0.3.x

 **Important: This tutorial targets langchain-core 1.2.16 (latest as of March 2026)**

Version 1.2.17 does not yet exist on PyPI. The closest available version is 1.2.16, which is used throughout this document and the companion notebook.

LangChain made a major version jump from 0.3.x to 1.0 in late 2025, introducing several breaking changes. If you are migrating from older code, see the table below.

Area	v0.3.x (old—DO NOT USE)	v1.x (current—USE THIS)
Pydantic imports	<code>from langchain_core.pydantic_v1 import BaseModel</code>	<code>from pydantic import BaseModel</code>
Agents	<code>create_react_agent + AgentExecutor</code>	<code>from langchain.agents import create_agent</code>
Agent prompt param	<code>state_modifier=</code> or <code>prompt=</code>	<code>system_prompt=</code>
Message text access	<code>.text()</code> - method call	<code>.text</code> —property (no parentheses)
AIMessage extras	<code>.example</code> parameter	use <code>.additional_kwargs</code>
LangGraph dependency	separate install	bundled with langchain 1.x

Introduction: What Is LangChain and Why Does It Matter?

Imagine building a customer service chatbot. Without LangChain, you would manually write code to call the LLM API, manage conversation history, format prompts, parse JSON responses, handle errors, retry on failure, stream output, and connect a vector database. That is hundreds of lines of boilerplate and it grows with every feature you add.

LangChain solves this with a unified set of composable building blocks. You focus on your application logic; LangChain handles the plumbing.



LangChain in One Sentence

LangChain is an open-source Python framework providing composable building blocks for connecting LLMs to tools, memory, data, and each other with production-grade reliability.

The LangChain Ecosystem (v1.x)

Package	What It Contains	When You Need It
langchain-core 1.2.16	Base abstractions: Runnable, Messages, Prompts, Parsers	Always — the foundation
langchain 1.2.10	High-level agents, memory wrappers, chains	Most applications
langchain-groq	Groq provider (free Llama 3 inference)	When using Groq/Llama 3
langchain-community	Loaders, vector stores, tools	Extra connectors
langchain-huggingface	HuggingFace embeddings (free, no API key)	For local embeddings/RAG
langchain-text-splitters	Document chunking utilities	For RAG applications

Feature	langchain-core	langchain
Primary Role	The foundation/base layer.	

The Central Concept: Everything Is a Runnable

The most important idea in modern LangChain is the interface. Every component — LLM, prompt template, parser, retriever, custom function — exposes the same four methods:

The Runnable Interface (same for every component)

```
runnable.invoke(input)           # single input → single output
runnable.batch([input1, input2]) # multiple inputs → multiple outputs (parallel)
runnable.stream(input)           # single input → token-by-token stream
await runnable.ainvoke(input)     # async version of invoke
```

Because every component implements the same interface, they can be composed with the pipe operator — LangChain Expression Language (LCEL).

Module 2 — LLM Basics

The Three Message Types

Class	Role	Analogy	When to Use
SystemMessage	Sets AI persona	Job description	Once, at the start of every conversation
HumanMessage	User input	The question	Every user turn
AIMessage	AI reply	The answer	Stored in history; returned by the LLM

⚠️ v1.x Change: .text is Now a Property

In v0.3.x you called `response.text()` with parentheses.

In v1.x, `.text` is a property: `response.text` (no parentheses).

Using the old method form still works but emits a `DeprecationWarning`.

Notebook: Module 2 – Invoking the LLM

```
from langchain_core.messages import HumanMessage, SystemMessage

response = llm.invoke([
    SystemMessage(content="You are a professional chef."),
    HumanMessage(content="How do I stop onions from making me cry?"),
])

print(response.text)          # ← v1.x: property, not method
print(response.usage_metadata) # token count – track your costs
```

Module 3 — Prompt Templates

Why Use Templates?

- Separate prompt structure from dynamic data — same structure, different inputs
- Test and version-control prompts as code
- Use `.partial()` to pre-fill variables and build reusable prompt families
- `FewShotChatMessagePromptTemplate` shows the AI examples of the exact output format

Notebook: Module 3 – ChatPromptTemplate

```
# Variables wrapped in {curly_braces} are filled at invoke time
product_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a copywriter. Write {tone} product descriptions."),
    ("human", "Product: {product_name}. Features: {features}"),
])

chain = product_prompt | llm | StrOutputParser()
result = chain.invoke({
    "tone": "enthusiastic",
    "product_name": "Wireless Headphones",
    "features": "40hr battery, foldable",
})
```

Module 4 — LCEL: LangChain Expression Language

The Five Key Runnables

Runnable	What It Does	Analogy
(RunnableSequence)	Chains steps: output of one → input of next	Assembly line
RunnableParallel	Runs multiple chains on the same input simultaneously	Parallel workers
RunnableBranch	Routes to different chains based on a condition	if/elif/else
RunnableLambda	Wraps any Python function into a Runnable	Adapter
RunnablePassthrough.assign()	Passes input through and adds new computed fields	Enrich / augment

RunnableParallel — Why It Matters for Performance



Sequential vs Parallel Execution

Sequential (old approach): Title(1s) + Meta(1.5s) + Keywords(0.8s) = 3.3 seconds total

Parallel (RunnableParallel): MAX(1s, 1.5s, 0.8s) = 1.5 seconds total

For N independent tasks, RunnableParallel is up to N times faster.

Notebook: Module 4 – SEO Tool (3 tasks at once)

```
seo_pipeline = RunnableParallel(  
    title=title_chain,  
    meta_description=meta_chain,  
    keywords=keywords_chain,  
)  
# All three LLM calls fire simultaneously  
result = seo_pipeline.invoke({"topic": "Remote work best practices"})
```

RunnableBranch — Dynamic Routing

Think of RunnableBranch as a triage system. An incoming message is classified first, then routed to the right specialist. This creates intelligent pipelines that adapt to input type without any manual if/else in your application code.

Notebook: Module 4 – Customer Support Router

```
router = RunnableBranch(  
    (lambda x: "BILLING" in x["category"], billing_specialist_chain),  
    (lambda x: "TECHNICAL" in x["category"], tech_support_chain),  
    general_support_chain, # default if no condition matches  
)
```


Module 5 — Output Parsers (Native Pydantic v2)

The Most Important v1.x Change for Parsers

● Breaking Change: Pydantic Imports

In v0.3.x: `from langchain_core.pydantic_v1 import BaseModel, Field`

In v1.x: `from pydantic import BaseModel, Field`

`langchain_core.pydantic_v1` has been REMOVED in v1.x. Any code using it will fail.

LangChain now uses native Pydantic v2 throughout. Update all your models.

PydanticOutputParser — Strongly Typed LLM Output

Define a Pydantic v2 model. The parser auto-generates format instructions and validates the LLM response into a real typed Python object.

Notebook: Module 5 — Job Posting Extractor (Pydantic v2)

```
from pydantic import BaseModel, Field # ← native pydantic v2, NOT pydantic_v1
from typing import List
```

```
class JobPosting(BaseModel):
    title: str = Field(description="Job title")
    company: str = Field(description="Company name")
    salary_range: str = Field(description="Salary range")
    required_skills: List[str] = Field(description="Required skills")
    is_remote: bool = Field(description="Is remote?")
```

```
parser = PydanticOutputParser(pydantic_object=JobPosting)
chain = extraction_prompt | llm | parser
```

```
# job is a typed Python object:
# job.title, job.required_skills, job.is_remote — not dict keys!
```

Parser	Returns	Best For
StrOutputParser	str	Simple text, no structure needed
JsonOutputParser	dict or list	Flexible structured data
PydanticOutputParser	Pydantic model instance	Strongly typed, validated objects

Module 6 — Memory & Chat History

Why LLMs Have No Memory

Every LLM call is completely stateless. When you send message #5, the model has no idea messages #1-#4 ever happened. Memory is an illusion created by the application secretly re-sending conversation history with every request.

formalizes this pattern — it automatically reads history from a session store before invoking the chain, and writes the new exchange back after.

Notebook: Module 6 – Stateful Chatbot

```
# 1. Prompt with a history slot
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful coding assistant."),
    MessagesPlaceholder(variable_name="history"), # injected automatically
    ("human", "{input}"),
])

# 2. Wrap with history management
chatbot = RunnableWithMessageHistory(
    base_chain,
    get_session_history, # function: session_id → history object
    input_messages_key="input",
    history_messages_key="history",
)

# 3. Same session_id = shared memory across turns
config = {"configurable": {"session_id": "user-abc"}}
chatbot.invoke({"input": "My name is Alex."}, config=config)
chatbot.invoke({"input": "What was my name?"}, config=config) # remembers Alex
```

Token Budget Warning

Every message stored in history costs tokens. A 100-turn conversation may exceed the model context window. Use Windowed Memory (keep last K turns) to prevent this. See the WindowedHistory class in the notebook for a practical implementation.

Module 7 — RAG: Retrieval-Augmented Generation

The Problem RAG Solves

LLMs have a training cutoff and know nothing about your private data. RAG gives the LLM a searchable reference library at query time — the model can only answer from what is retrieved, dramatically reducing hallucination.



RAG Explained Simply

Without RAG: a doctor answers from memory alone.

With RAG: the doctor looks up the latest research papers first, then answers.

RAG = give the LLM a searchable reference library it can consult before responding.

The 4-Step RAG Pipeline

Step	What Happens	Class / Tool Used
1. Load	Read docs from PDFs, websites, databases	TextLoader, PyPDFLoader, WebBaseLoader
2. Split	Break large docs into small overlapping chunks	RecursiveCharacterTextSplitter
3. Embed + Store	Convert chunks to vectors; index in FAISS	HuggingFaceEmbeddings + FAISS
4. Retrieve + Generate	Search relevant chunks; LLM answers using context	VectorStoreRetriever + RAG chain

Chunk Size Tuning Guide

Setting	Too Small	Too Large	Recommended Start
chunk_size	More chunks, very focused but may miss context	Fewer chunks, more context per chunk	400-500 chars
chunk_overlap	Risk of losing info at boundaries	Duplicate content, higher storage	40-100 chars
k (retrieve)	Very precise but may miss relevant content	More context but higher token cost	2-4 chunks

Notebook: Module 7 – Full RAG Chain

```
# Flow: question → retrieve → format context → prompt → LLM → answer
rag_chain = (
    {
        "context": itemgetter("question") | retriever | format_docs,
        "question": itemgetter("question"),
    }
    | rag_prompt
```

```
| llm  
| StrOutputParser()  
)
```

```
# The LLM answers ONLY from retrieved company policy documents  
# – grounded, not hallucinated
```

Module 8 — Tools & Agents (v1.x API)

The New create_agent API

● Breaking Change: Agents in v1.x

v0.3.x used: `from langgraph.prebuilt import create_react_agent` (DEPRECATED)

v0.3.x used: `AgentExecutor(agent=..., tools=...)` (DEPRECATED)

v1.x uses: `from langchain.agents import create_agent`

v1.x uses: `system_prompt=` parameter instead of `prompt=` or `state_modifier=`

The new API is simpler, more reliable, and built on LangGraph internally.

Chains vs. Agents: When to Use Which

Dimension	Chains	Agents
Execution flow	Fixed, predefined sequence	Dynamic — LLM decides at runtime
Predictability	High — same input, same path	Lower — non-deterministic routing
Flexibility	Low	High
Best for	Structured, repeatable workflows	Open-ended, exploratory tasks
Real example	Invoice processing pipeline	Research assistant, portfolio calculator

Building a Tool with @tool

The `@tool` decorator turns any Python function into a LangChain Tool. The docstring is what the agent reads to decide when and how to use the tool. Write specific, clear docstrings — vague ones cause wrong tool calls.

Notebook: Module 8 – Custom Calculator Tool

```
@tool
def calculate(expression: str) -> str:
    """
    Safely evaluate a math expression. Use for arithmetic, percentages,
    or any calculation. Input: valid Python math expression.
    Examples: "2 + 2", "85.50 * 1.18", "math.sqrt(144)"
    """
    ...

# The agent reads the docstring to decide WHEN to call this tool
```

The New Agent in Practice

Notebook: Module 8 – create_agent (v1.x)

```
from langchain.agents import create_agent # ← new v1.x import

agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt="You are a helpful financial assistant...", # ← new param name
)

# invoke returns a dict with a "messages" list
result = agent.invoke({
    "messages": [HumanMessage(content="What is 18% tip on $92.40?")]
})

# Final answer is the last message
print(result["messages"][-1].content)
```



Agent Safety Best Practices

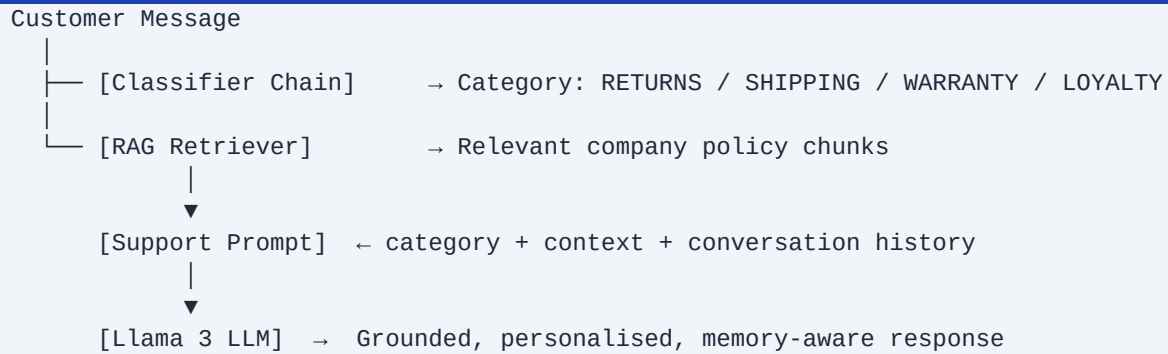
Agents are non-deterministic — always test thoroughly before production.
For tools that modify state (DB writes, emails), add a human approval step.
Use `agent.stream(...)` to see each step and detect problems early.
Keep tools focused with clear descriptions — fewer, better tools beats many vague ones.

Module 9 — Capstone: Full Customer Support Assistant

Architecture

This capstone assembles everything from the previous modules into one coherent production-grade system. The design is used in real customer support bots today.

System Architecture Diagram



Why Each Component Is There

Component	Why It Is Needed	Without It
Classifier	Tells the LLM what type of question is coming	Generic, unfocused responses
RAG context	Grounds the LLM in actual company policy	LLM may hallucinate wrong policies
Conversation history	Allows natural follow-up questions	Customer must repeat themselves
System persona	Consistent brand voice and tone	Unpredictable, off-brand replies

Notebook: Module 9 – Pipeline Assembly

```
# Augment input with category + RAG context in parallel
augmented = RunnablePassthrough.assign(
    category=itemgetter("input") | classifier_chain,
    context=itemgetter("input") | retriever | format_docs,
)

# Wrap with memory for stateful multi-turn conversation
support_bot = RunnableWithMessageHistory(
    augmented | support_prompt | llm | StrOutputParser(),
    get_session_history,
    input_messages_key="input",
    history_messages_key="history",
)
```

Quick Reference: When to Use What

I want to...	Use this	Module
Call an LLM	<code>llm.invoke([HumanMessage(...)])</code>	2
Stream output token by token	<code>llm.stream(...)</code>	2
Build a reusable prompt	<code>ChatPromptTemplate.from_messages(...)</code>	3
Show the LLM examples	<code>FewShotChatMessagePromptTemplate</code>	3
Build a sequential pipeline	<code>prompt llm parser</code>	4
Run tasks simultaneously	<code>RunnableParallel(a=chain1, b=chain2)</code>	4
Route by condition	<code>RunnableBranch(condition, chain, default)</code>	4
Add computed fields to state	<code>RunnablePassthrough.assign(...)</code>	4
Extract structured Python object	<code>PydanticOutputParser (pydantic v2)</code>	5
Remember conversation turns	<code>RunnableWithMessageHistory</code>	6
Q&A over private documents	FAISS + HuggingFaceEmbeddings + RAG chain	7
LLM chooses tools dynamically	<code>from langchain.agents import create_agent</code>	8
Build a full support bot	Combine all the above	9

Next Steps

Going to Production

- LangSmith — trace, test, and evaluate every invocation in production
- Replace FAISS with Chroma or Pinecone for persistent, scalable vector storage
- Replace InMemoryChatMessageHistory with Redis or PostgreSQL
- Use `runnable.with_retry()` and `runnable.with_fallbacks([...])` for resilience

Advanced Topics in v1.x

- Agent Middleware — new v1.x feature to intercept and modify agent execution steps
- LangGraph — stateful, graph-based multi-agent workflows with complex branching
- Structured output strategies: ToolStrategy, ProviderStrategy in `create_agent`
- Multi-vector retrieval and reranking for higher-accuracy RAG

Key Links

- LangChain GitHub: github.com/langchain-ai/langchain
- v1.x Migration Guide: docs.langchain.com/oss/python/migrate/langchain-v1
- Groq Free Tier: console.groq.com (Llama 3 at no cost)
- PyPI langchain-core: pypi.org/project/langchain-core/